



VIT[®]

BHOPAL

Project Title: Mess Management
System

Course Name: Programming in
java

Submitted by: Omkar Kumar

Registration Number:

25bai10893

Introduction



The mess Management System is a desktop application built to bring local mess into the digital age. It replaces paper based student tracking with a secure MySQL database system. The application features a Java-based backend for processing logic, a command-line interface for rapid admin tasks, and a Java Swing graphical dashboard to view active student enrollments in real time.

Problem statement

When I looked at the local hostels and mess facilities around Ashtha, I saw that the admins are still doing everything by hand. They use physical registers to track who is eating lunch or dinner, and figuring out the monthly bills takes forever. Because it's all on paper, its really easy to make mistakes or lose the data entirely. I wanted to build a simple digital solution to fix this exact problem and make their daily work a lot easier.

Function Requirements

1. **Student enrollment :** The system allows admins to add new students, view existing records, update their meal plans, and delete students who leave the mess.
2. **Meal plan tracking:** Mess manager can select specific meal plans for students (Lunch only, Dinner only, or Both).
3. **Automated billing generation:** The system calculates the total monthly fee based on the active meal plan and generates a physical .txt receipt using Java File I/O.

4.Live dashboard: A graphical user interface (GUI) that instantly pulls and displays the current student roster from the database.

Non-function Requirements

1. **Reliability:** The system uses a persistent MySQL database, meaning data won't be lost if the application is closed or the computer crashes (unlike array-based storage).
2. **Usability:** The interface is split between a fast, text-based terminal for data entry and a simple graphical table for viewing, making it easy for non-technical mess staff to use.
3. **Maintainability:** The code is modularized into separate files (e.g., UI, Database Connection, Main Logic) so future updates won't break the whole system.
4. **Error Handling:** The program includes custom exceptions to catch things like duplicate student IDs, preventing the database from throwing fatal errors and crashing the app.

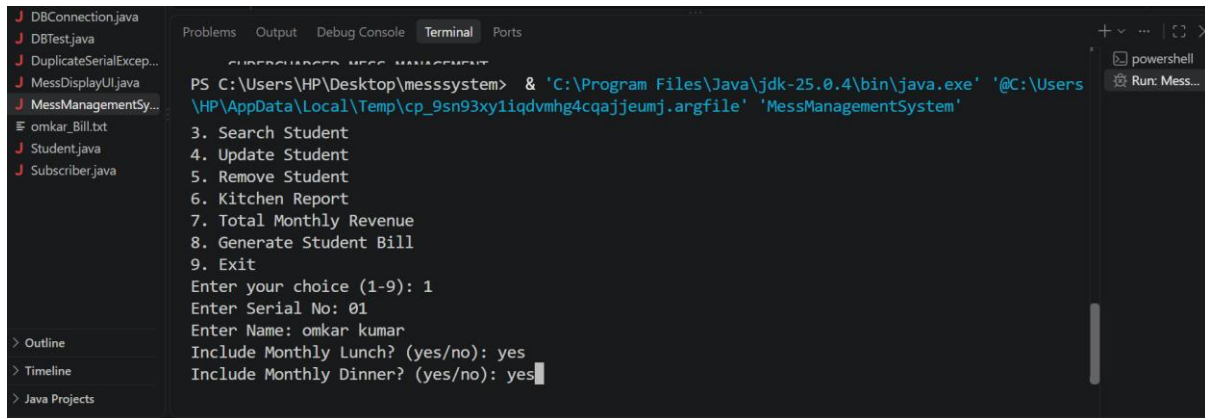
System Architecture

The project uses a standard Two-Tier Desktop Architecture:

- **Client Tier (Frontend):** Java Swing (for the dashboard) and a Java Console (for the admin inputs).
- **Data Tier (Backend):** A local MySQL database. The Java application communicates directly with this database using the JDBC API (mysql-connector-j)

Design Diagram

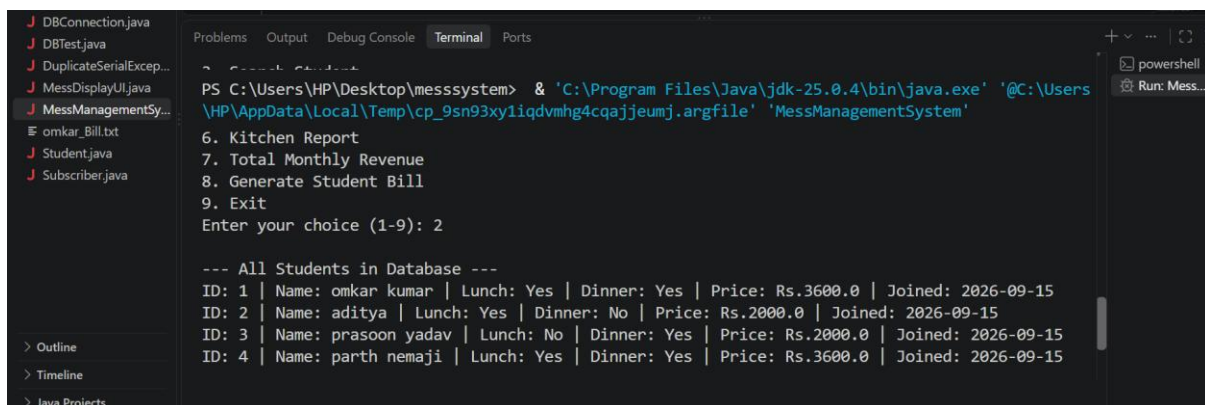
1.To add student



```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'
```

3. Search Student
4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 1
Enter Serial No: 01
Enter Name: omkar kumar
Include Monthly Lunch? (yes/no): yes
Include Monthly Dinner? (yes/no): yes

2.view all student



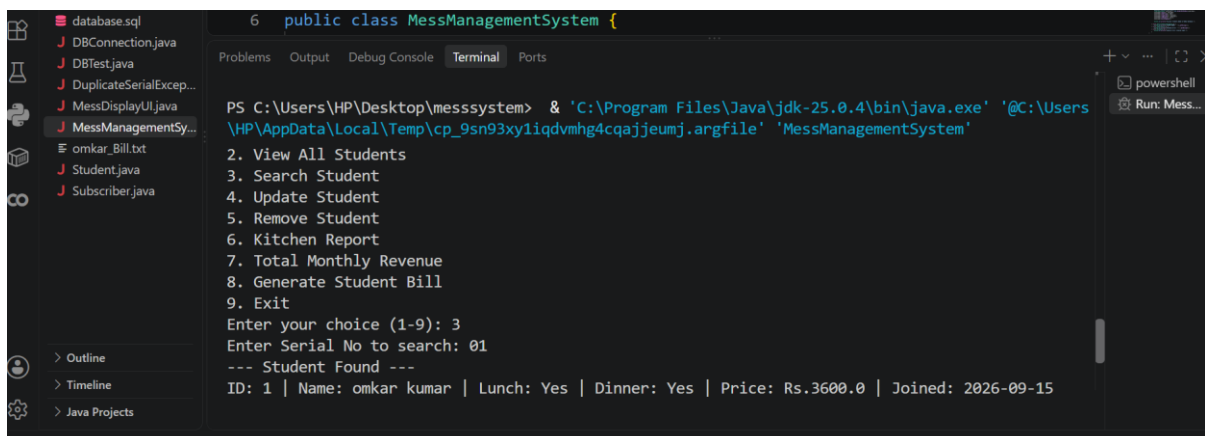
```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'
```

6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 2

--- All Students in Database ---

ID: 1	Name: omkar kumar	Lunch: Yes	Dinner: Yes	Price: Rs.3600.0	Joined: 2026-09-15
ID: 2	Name: aditya	Lunch: Yes	Dinner: No	Price: Rs.2000.0	Joined: 2026-09-15
ID: 3	Name: praseon yadav	Lunch: No	Dinner: Yes	Price: Rs.2000.0	Joined: 2026-09-15
ID: 4	Name: parth nemaji	Lunch: Yes	Dinner: Yes	Price: Rs.3600.0	Joined: 2026-09-15

3.search any student

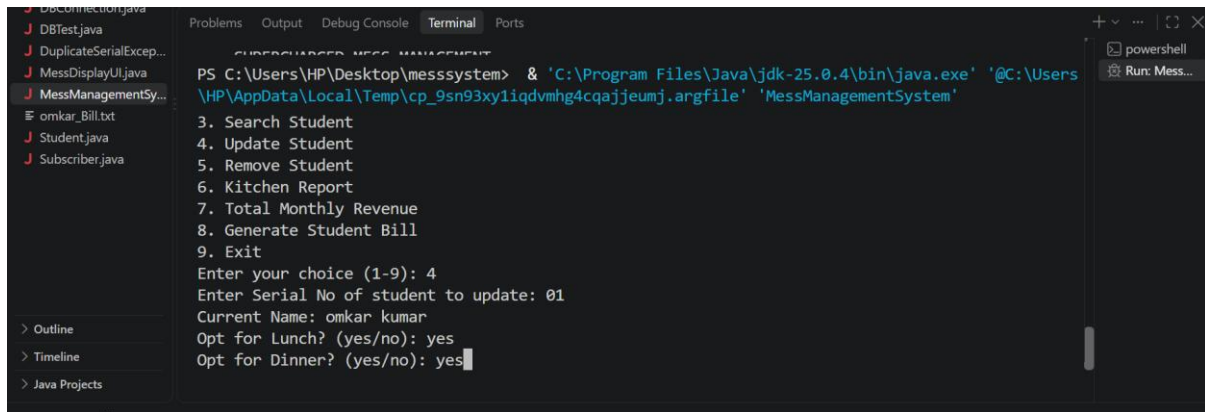


```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'
```

2. View All Students
3. Search Student
4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 3
Enter Serial No to search: 01
--- Student Found ---

ID: 1	Name: omkar kumar	Lunch: Yes	Dinner: Yes	Price: Rs.3600.0	Joined: 2026-09-15
-------	-------------------	------------	-------------	------------------	--------------------

4.update any student data

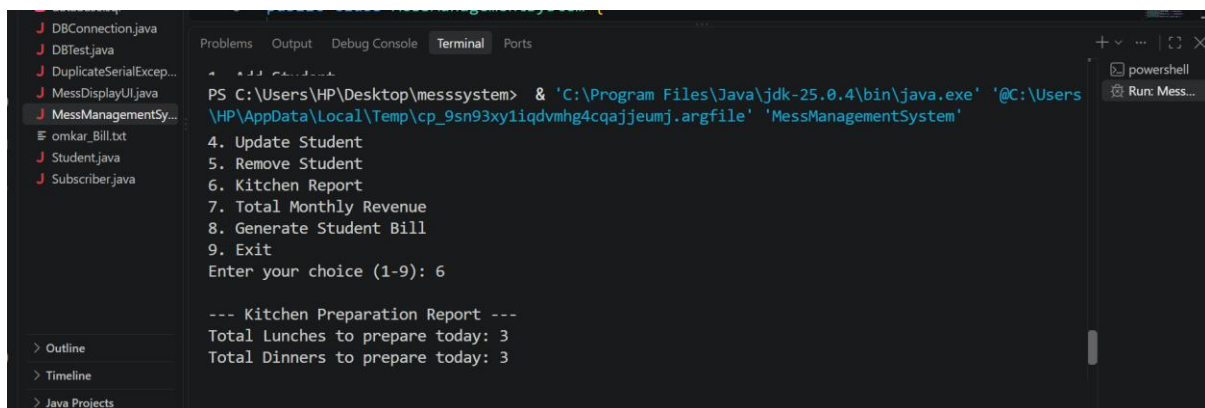


```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'

SUBSCRIBER MESS MANAGEMENT

3. Search Student
4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 4
Enter Serial No of student to update: 01
Current Name: omkar kumar
Opt for Lunch? (yes/no): yes
Opt for Dinner? (yes/no): yes
```

5.kitchen report

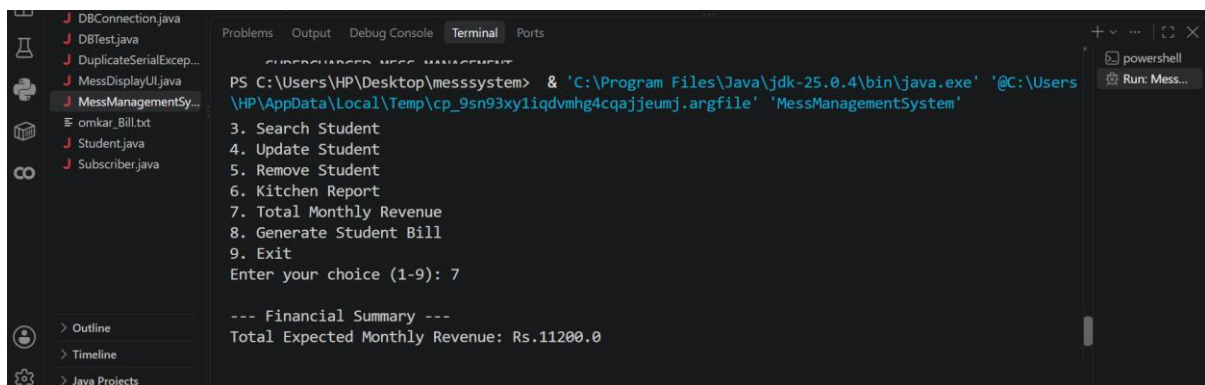


```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'

4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 6

--- Kitchen Preparation Report ---
Total Lunches to prepare today: 3
Total Dinners to prepare today: 3
```

6.Total monthly revenue



```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'

3. Search Student
4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 7

--- Financial Summary ---
Total Expected Monthly Revenue: Rs.11200.0
```

7.Generate any student bill

```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'

=== SUPERCHARGED MESS MANAGEMENT ===

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 8
Enter Serial No of student to generate bill: 01
```

8.To exit the system

```
PS C:\Users\HP\Desktop\messsystem> & 'C:\Program Files\Java\jdk-25.0.4\bin\java.exe' '@C:\Users\HP\AppData\Local\Temp\cp_9sn93xy1iqdvmhg4cqajjeumj.argfile' 'MessManagementSystem'

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Remove Student
6. Kitchen Report
7. Total Monthly Revenue
8. Generate Student Bill
9. Exit
Enter your choice (1-9): 9
Exiting System... All data is safely stored in MySQL.
PS C:\Users\HP\Desktop\messsystem>
```

9.Bill

```
3 =====
4 Student Name: omkar
5 Serial No: 1
6 Join Date: 2026-09-12
7 Lunch Included: Yes
8 Dinner Included: Yes
9 =====
10 TOTAL AMOUNT DUE: Rs.3600.0
11 =====
12
```

10. Java Gui

ID	Name	Lunch	Dinner	Price (Rs)	Join Date
1	omkar	Yes	Yes	3600.0	2026-09-12
2	aditya	Yes	No	2000.0	2026-09-15
3	praseon yadav	No	Yes	2000.0	2026-09-15
4	parth nemaji	Yes	Yes	3600.0	2026-09-15

11.MySQL

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

messdb

messmanagementdb

Tables

students

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

sys

SQL File 3*

name VARCHAR(100),

includeLunch BOOLEAN,

includeDinner BOOLEAN,

monthlyPrice DOUBLE,

joinDate DATE DEFAULT (CURRENT_DATE)

);

SELECT * FROM students;

Result Grid

serialNo	name	includeLunch	includeDinner	monthlyPrice	joinDate
1	omkar kumar	1	1	3600	2026-09-14
2	aditya	1	0	2000	2026-09-14
3	praseon yadav	0	1	2000	2026-09-14
4	parth nemaji	1	1	3600	2026-09-14
total		total	total	total	total

students 4 x

Apply Revert Context Help Snippets

Output

#	Time	Action	Message	Duration / Fetch
18	22:44:33	CREATE TABLE IF NOT EXISTS students (serialN...	0 row(s) affected, 1 warning(s): 1050 Table 'students' a...	0.000 sec
19	22:44:33	SELECT * FROM students LIMIT 0.1000	4 row(s) returned	0.000 sec / 0.000 sec
20	22:46:05	CREATE DATABASE IF NOT EXISTS MessManagem...	1 row(s) affected, 1 warning(s): 1007 Can't create data...	0.015 sec
21	22:46:05	USE MessManagementDB	0 row(s) affected	0.000 sec
22	22:46:05	CREATE TABLE IF NOT EXISTS students (serialN...	0 row(s) affected, 1 warning(s): 1050 Table 'students' a...	0.000 sec
23	22:46:05	SELECT * FROM students LIMIT 0.1000	4 row(s) returned	0.000 sec / 0.000 sec

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Design decision & rationale

- **Why Java and Swing?** I chose Java because of its strong Object-Oriented principles. Swing was chosen for the dashboard because it is built into the JDK and doesn't require complex web server setups for a simple local application.
- **Why MySQL?** I needed a permanent way to store data. Moving away from standard Java arrays to a relational database ensures the mess data is kept safe and can be easily queried.
- **Why File I/O for Receipts?** I decided to generate .txt files for billing so that the mess admin has a physical file they can easily print and hand to the student.

Implementation Detail

The project was coded in Java (JDK 8+) using VS Code. The database was designed in MySQL Workbench. I used the `java.sql` package to handle the database connections and `javax.swing` for the UI components. The classpath was configured to include the external `mysql-connector-j-26.7.0.jar` driver to bridge the Java code with the local MySQL server.

Testing approach

I used manual functional testing for this project. I tested the CRUD operations by inputting fake student data and verifying it appeared correctly in both the Java GUI and the MySQL Workbench. I also performed boundary testing by purposely trying to add a student with an ID that already existed to ensure my exception handling caught the error instead of crashing.

Challenges faced

Due to unfamiliarity with MySQL ,one of the biggest hurdles that I faced was linking the MySQL database to the Java code. Initially, my IDE could not find the database driver, which resulted in `ClassNotFoundException` exceptions. I had to learn how to properly configure the Java classpath and link the .jar library file. Transitioning the logic from temporary arrays to permanent SQL INSERT and SELECT queries was also a steep learning curve.

Learning and key takeaways

This project really helped me understand how front-end code interacts with back-end data. I learned how to write standard SQL queries and execute them dynamically from within Java using `PreparedStatement`. I also got comfortable with standardizing a project folder structure by separating `src/`, `lib/`, and `database/` folders.

Future Enhancement

If I were to expand this project later, I would add a secure Login/Authentication page for the admin. I would also like to implement an online payment integration so students could pay their monthly bills directly through the system instead of just generating a text receipt.

Reference

1. Oracle Java Documentation (File I/O and Swing API).
2. MySQL 8.0 Reference Manual.

3. Class lectures and notes on Object-Oriented Programming and JDBC connections.

4. vityarthi